Worked for 1m 24s

Yes. Given what v15 has become, I would **split the package line without splitting the SignalTree brand**.

My recommendation is:

```
SignalTree 14
    @signaltree/core@14.x
    Angular-specific legacy line
    patch/security/correctness maintenance only

SignalTree 15
    @signaltree/kernel@15.x
        neutral truth kernel

    @signaltree/angular@15.x
        Angular realization/adapter
        depends on @signaltree/kernel
        normal Angular users start here
```

I would make both v15 packages **public, free, and Apache-2.0**. I would not make the kernel private, and I would not switch it to MIT.

Why I prefer a new package over `@signaltree/core@15`

Semver absolutely permits turning `@signaltree/core` into something radically different at v15. But in this case the product's meaning has changed enough that the package boundary itself has useful semantic information.

Today `@signaltree/core` explicitly presents itself as **"Reactive JSON for Angular"**, and its current mental model says leaves are Angular signals. GitHub +1

V15's model is becoming:

```
SignalTree state
    is NOT Angular state.

Angular signals
    are one realization of SignalTree state.
```

That's a much bigger distinction than an API cleanup.

So:

```
@signaltree/core
    = historical Angular SignalTree
```

```
@signaltree/kernel
    = actual v15 framework-neutral state authority

@signaltree/angular
    = Angular's view/realization of that authority
```

That makes the package topology teach the architecture.

And your proposed legacy policy is exactly what I would do:

> Freeze feature development on `@signaltree/core@14`, keep a v14 maintenance branch, and issue 14.x patches for users who don't want the migration.

I would give that line a defined support promise—something like correctness/security fixes for 12–18 months after v15 GA rather than "forever."

One thing I would *not* do

I wouldn't create an entirely new project/repository/brand.

Keep:

```
SignalTree
signaltree.io
github.com/.../signaltree
@signaltree/*
```

You're changing the architecture, not abandoning the lineage.

I'd also version the new packages **15.0.0**, not 1.0.0:

```
@signaltree/kernel@15.0.0
@signaltree/angular@15.0.0
```

Even though they're technically new npm packages, they're components of SignalTree 15. That makes docs, compatibility tables, adapters, and migration much easier to understand.

Which package should normal developers install?

I would make `@signaltree/angular` the normal Angular-facing install.

For example:

```
TypeScript
```

```
import { signalTree } from '@signaltree/angular';
```

`@signaltree/angular` can re-export appropriate kernel API rather than redeclaring it.

Underneath:

```
@signaltree/angular
        |
        └── @signaltree/kernel
```

A framework-neutral/worker/embedded user instead does:

TypeScript

```
import { signalTree } from '@signaltree/kernel';
```

This creates a very useful distinction.

You don't want every Angular user thinking:

Bash

```
npm i @signaltree/kernel @signaltree/angular
```

unless technically unavoidable.

Prefer:

Bash

```
npm i @signaltree/angular
```

and let npm bring kernel along.

Advanced/frameworkless users explicitly choose kernel.

Apache-2.0, not MIT

I would **keep Apache-2.0**.

Current `@signaltree/core` has already moved to Apache-2.0, and the project explicitly calls out the patent grant and trademark separation. GitHub

MIT would give you basically no adoption advantage here.

Apache gives you the permissiveness people want **plus an explicit patent license and patent-retaliation provision**. For something that could become foundational infrastructure in commercial applications, that's valuable.

So:

```
@signaltree/kernel      Apache-2.0
@signaltree/angular     Apache-2.0
future React adapter    Apache-2.0
public SDK/contracts    Apache-2.0
```

I would particularly avoid licensing adapters differently from the kernel. You want third parties to be comfortable building against those contracts.

Also important: trademark

Open-source the **code**, retain the **SignalTree brand**.

Apache does not grant rights to your trademarks, which your current project documentation already notes. GitHub

That's a useful open-source commercial model:

```
Anyone can fork the kernel.

They cannot present their fork as official SignalTree.
```

I would not make the kernel private

I think that would damage the opportunity you're uncovering.

A private foundational state dependency creates exactly the wrong friction:

```
enterprise security review
source visibility
framework adapter development
AI/agent tooling
third-party integrations
community experimentation
academic/research use
worker/runtime integrations
```

The more SignalTree moves toward a **multi-runtime state substrate**, the more valuable public contracts become.

There is also an adoption trust problem.

Teams are much more willing to build their application's state architecture around:

```
Apache-licensed package
public source
auditable invariants
portable data model
```

than:

```
proprietary npm dependency
whose vendor can change commercial terms
```

Especially for a state system explicitly selling them **less lock-in**.

A proprietary neutral kernel would create an odd message:

> “We're freeing your state from framework lock-in, but putting it behind vendor lock-in.”

I wouldn't do that.

Free or sold?

I would not sell access to the kernel.

I'd use:

Open source as distribution

```
FREE / APACHE

@signaltree/kernel
@signaltree/angular
identity semantics
held references
Link contracts
causal semantics
core history
entity semantics
official basic adapters
```

Those are the differentiated properties that make people care about SignalTree.

Don't hide the thing you're trying to establish as a standard.

Instead monetize **operational leverage around the substrate**.

There are several much better businesses hiding here.

1. Enterprise support

The simplest eventual model:

```
SignalTree
    free forever

SignalTree Enterprise Support
    migration assistance
    architecture reviews
    upgrade guarantees
    priority bug handling
    SLA
    long-term support versions
```

Companies routinely pay for certainty even when the dependency is Apache licensed.

You don't need a separate code fork to sell this.

---

2. SignalTree Studio / advanced DevTools

The causal model is unusually suited to a serious developer product.

Imagine:

```
WHO changed this subject?
WHY did this consequence happen?
WHAT causal turn produced it?
WHAT would undo reverse?
WHERE did this realization originate?
WHICH held references still point here?
WHAT was inspection-only?
```

That can become much more interesting than Redux DevTools.

Something like:

```
SignalTree kernel        free
basic DevTools           free

SignalTree Studio
    causal graph visualization
    timeline/replay
```

```
AI-agent action inspection
team traces
saved debugging sessions
remote diagnostics
```

could be paid.

The kernel itself remains open.

---

3. Hosted synchronization / collaboration

This may eventually be the largest commercial opportunity if the architecture continues toward AI/multi-actor applications.

```
browser
    |
SignalTree kernel
    |
Link
    |
SignalTree Relay / Cloud
    |
other browser
worker
AI agent
server
```

You can open-source the client protocol and charge for the hosted infrastructure.

That gives you an actual recurring-service moat instead of trying to charge developers for 10 KB of JavaScript.

---

4. Enterprise AI/agent observability

There's another plausible paid layer:

```
agent writes
tool calls
human writes
server realizations
rollbacks
causal traces
```

The state model could make those things inspectable in a way traditional frontend telemetry isn't.

That could eventually become:

```
SignalTree Agent Trace
SignalTree Audit
SignalTree Replay
```

But I would prove the use case before naming/productizing any of it.

What about BUSL?

You already have precedent for mixed licensing: for example, the currently published `@signaltree/schema` is listed as BUSL-1.1, while current core is Apache-2.0. npm +1

I would eventually simplify that story.

My preferred policy would become:

```
FOUNDATIONAL CLIENT LIBRARIES
    Apache-2.0

COMMERCIAL SERVER PRODUCTS
    proprietary / commercial

OPTIONAL PREMIUM CLIENT TOOLING
    commercial or BUSL if useful
```

I would **not** put `@signaltree/kernel` under BUSL.

BUSL can work well when you're protecting a server product from a cloud competitor. It's much less attractive for a client-side infrastructure dependency where adoption is the first battle.

And it isn't OSI open source.

Why not charge per developer for SignalTree itself?

Because I think you'd trade the bigger opportunity for the smaller one.

Imagine two outcomes.

Proprietary library

```
500 developers
× $20/month
```

```
= $120k ARR
```

Sounds nice.

But every prospective user has to ask:

```
Can legal approve it?
What if pricing changes?
What happens if SignalTree disappears?
Can we fork it?
Can our internal framework adapter inspect internals?
```

Those questions crush adoption.

Open substrate

Instead you're optimizing for:

```
10,000+
100,000+
1,000,000+
applications eventually depending on the semantic model
```

Then even a very small percentage buying:

```
support
cloud
team tooling
enterprise observability
hosted sync
migration
```

is a much stronger business.

SignalTree's unusual advantage is potentially becoming an **architectural substrate**.

Substrates benefit enormously from being open.

There is one downside to abandoning `@signaltree/core`

You lose some npm continuity.

The current package already has 141 published versions and all of the existing README/search/AI-training/disambiguation history points at `@signaltree/core`. npm

So migration communication needs to be very deliberate.

I'd do this at v15 GA:

```
@signaltree/core README


╔═══════════════════════════════════════════╗
║ SignalTree 14 — Legacy Angular line       ║
║                                           ║
║ Maintained for fixes through <date>.      ║
║                                           ║
║ New projects:                             ║
║   npm install @signaltree/angular         ║
║                                           ║
║ Framework-neutral:                        ║
║   npm install @signaltree/kernel          ║
║                                           ║
║ Migration: signaltree.io/migrate/v15      ║
╚═══════════════════════════════════════════╝
```

And signaltree.io should default to v15 with a prominent:

```
Docs:
    v15
    v14 legacy
```

Do **not** destroy the old docs.

I'd also use npm tags deliberately

For example:

```
@signaltree/core
    latest -> 14.latest
    legacy -> 14.latest
```

Since no v15 of that package exists, `latest` naturally remains v14.

For the new line:

```
@signaltree/kernel
    next   -> 15.0.0-rc.*
    latest -> 15.0.0 when GA

@signaltree/angular
```

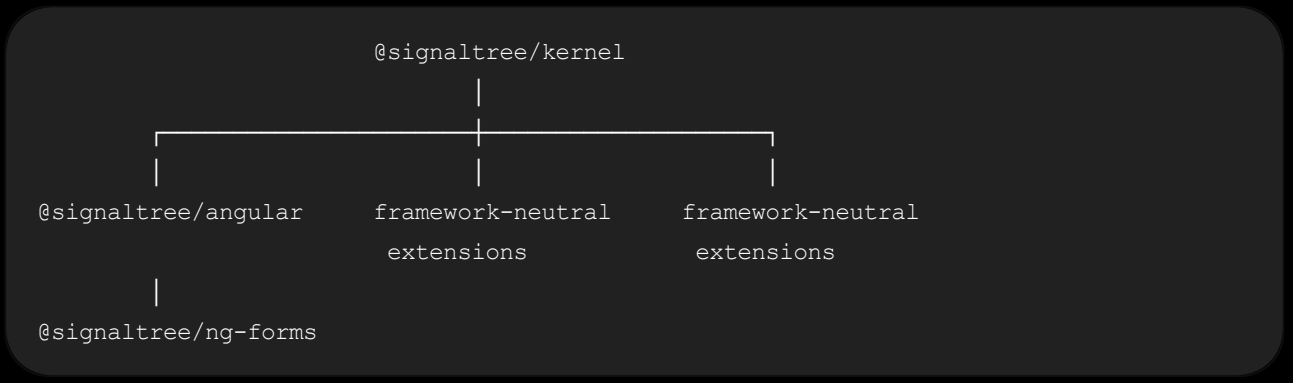
```
next   -> 15.0.0-rc.*
```

Before GA, I'd absolutely publish RCs under `next`, not `latest`.

Companion-package split

This new topology also gives us a useful test for the existing ecosystem.

Eventually:



Things like schema, realtime, events, serialization, etc. should not automatically depend on Angular merely because the old core did.

But I would **not perform that migration by taxonomy**.

As with everything else we've done:

> Measure who actually owns the semantic job.

My recommended commercial structure

If I were making the decision today:

Component	Visibility	License	Price
`@signaltree/kernel`	Public	Apache-2.0	Free
`@signaltree/angular`	Public	Apache-2.0	Free
future official framework adapters	Public	Apache-2.0	Free
fundamental identity/history/Link APIs	Public	Apache-2.0	Free

Component	Visibility	License	Price
basic local DevTools	Public	Apache-2.0	Free
enterprise support/LTS	Commercial	Contract	Paid
hosted synchronization/relay	Hosted	Proprietary service	Paid
advanced team/causal debugging	SaaS/commercial	Commercial	Paid
enterprise AI/agent auditing, if proven	SaaS/commercial	Commercial	Paid

I wouldn't try to monetize every row on that table immediately.

V15 itself should launch completely usable without money.

First goal:

```
adoption
credibility
ecosystem
real production usage
```

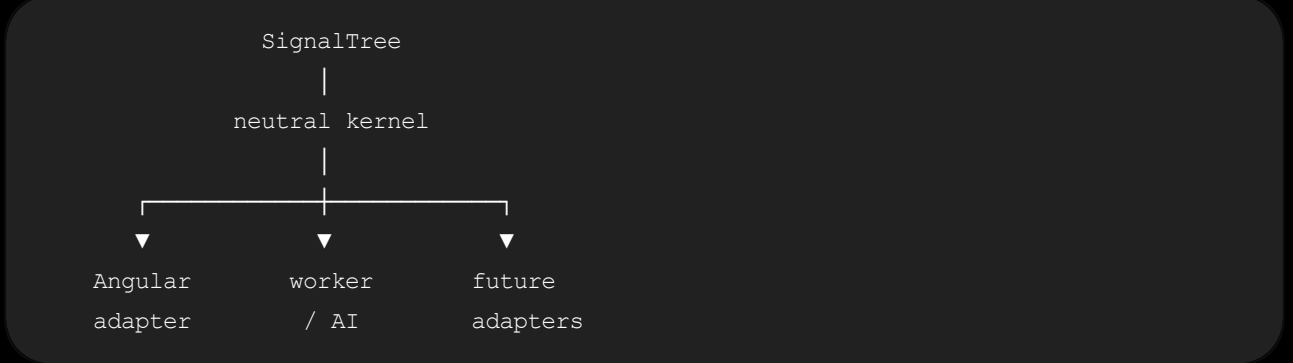
Then find what organizations actually ask you to operate for them.

The packaging decision also reinforces the product story

V14:

```
Angular state-management library
```

V15:



That's enough of an architectural break that `@signaltree/kernel` **is not merely a rename**.

It marks the moment SignalTree stopped being an Angular store with abstractions and became a neutral state substrate with Angular as one realization.

So yes: **I would follow your instinct—leave** `@signaltree/core@14` **alive as the legacy/LTS line, publish v15 as** `@signaltree/kernel` **+** `@signaltree/angular`**, keep the foundational packages Apache-2.0 and free, and monetize above the substrate rather than monetizing access to it.**

That gives v14 users stability without forcing them through this migration, while giving v15 permission to actually be the architecture we've spent all this effort proving.